

RCA-Agent

모니터링 기반 장애 원인 분석 에이전트의 설계 · 검증 · 개선

2026-08 · 개인 프로젝트 · Spring Boot · JDK 21 · Grafana Cloud(Tempo · Loki · Mimir) · Claude

실제 장애를 재현하고 측정하며, 관측 → 분석 → 평가의 병목을 단계적으로 찾아 개선했습니다.

Brave 기반 Trace와 Loki Log, Prometheus Metric을 연계한 RCA-Agent를 구축했습니다.

실제 장애를 재현해 Baseline과 채점 기준을 만들고, 반복적인 실험을 통해 Agent의 실패 원인을 추적했습니다. 그 결과 관측 데이터 부족, 분석 컨텍스트의 토큰 과다, 추론 결과의 판단 품질이라는 병목을 순차적으로 발견했고, 각 문제를 개선한 뒤 다시 평가했습니다.

재현 → 측정 → 분석 → 개선 → 재검증의 과정을 반복하며, 단순히 Agent의 성능을 높이는 것이 아니라 실험 결과를 근거로 다음 문제를 정의하고 해결하는 방식으로 시스템을 개선했습니다.

1. 문제 정의와 시스템 설계

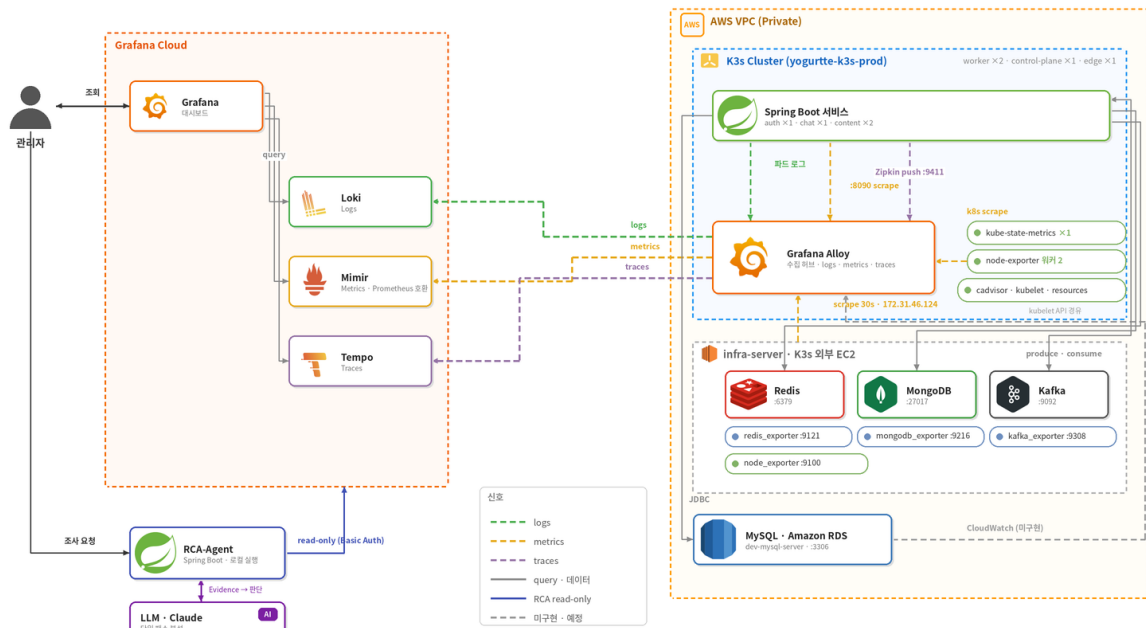
1.1 왜 만들었나

장애 조사는 순서가 정해져 있습니다. 시간대를 좁히고, Trace에서 이상 구간을 찾고, 같은 traceId의 Log를 붙이고, Metric으로 영향 범위를 확인합니다. 문제는 이 과정이 장애마다 반복되고, 세 신호의 관계를 매번 사람이 직접 연결해야 한다는 점입니다. 이 정형 작업을 자동화하되 목표를 "원인 맞히기"로 두지 않았습니다. Root Cause와 이를 뒷받침하는 Evidence를 함께 제시해야 실제 운영에서 쓸 수 있기 때문입니다. 검증할 수 없는 출력은 운영에 쓰지 못합니다.

1.2 아키텍처와 핵심 설계 원칙

입력은 두 가지입니다 - POST /investigate(자연어)와 POST /diagnose(traceId). Triage가 Trace(error · slow) / Log 광역 / Metric 이상의 3채널을 스캔해 Signal을 Incident 후보(시간창 · 대상 서비스 · 후보 traceId)로 만들고, Collector가 Tempo · Loki · Mimir에서 Evidence를 수집한 뒤, 단일 패스 LLM이 최종 Report를 냅니다.

관측(Observability) 파이프라인



[그림 1] 관측 파이프라인 - 3개 서비스가 Kafka로 연결되고, K3s와 외부 infra-server를 Grafana Alloy 하나로 모아 Tempo · Prometheus · Loki에 보낸다. 에이전트는 이 세 백엔드만 읽는다.

핵심 원칙은 "탐색은 결정론적으로, 판단은 LLM으로"입니다. 혼한 형태인 tool-calling 루프를 일부러 쓰지 않았습니다. "루프가 더 낫다"는 주장은 루프 없는 기준선이 있어야 증명되는데, 루프부터 시작하면 비교 대상이 없기 때문입니다.¹ 부수적으로 단일 패스는 실패를 셋으로 나눠 줍니다 - (a) 신호는 왔는데 모델이 놓침, (b) 신호가 수집되지 않음, (c) 보고도 확정 불가. 이 분류가 다음 투자를 프롬프트에 할지, 수집에 할지, 코드 연동에 할지를 정합니다. 결과적으로 이 문서의 대부분은 (b) 이야기입니다.

2. Observability 부터 검증했다

2.1 Brave vs OTel Agent - 전환 대신 실측

에이전트를 만들기 전에 현재 Trace가 장애 분석에 충분한지부터 확인했습니다. Kafka를 포함한 서비스 간 호출에서 계측 누락 가능성이 있어 OTel Agent 전환을 하나의 가설로 두었지만, 계측 기술을 먼저 바꾸는 대신 실제 E2E Trace를 측정했습니다. 확인할 것은 하나였습니다 - Kafka 경계를 넘어 동일 traceId가 실제로 이어지는가. 실측 결과 이어짐이 확인되어 48시간 A/B 테스트 없이 Brave 유지 · OTel A/B 미실행으로 의사결정을 단았습니다(ADR-001). 여기서 중요한 것은 어떤 기술을 골랐는가가 아니라, 전환

¹ 단일 패스 baseline - ADR-002(동일 트레이스 3회 입력 편차 $\pm 0.3\%$). 계측 방식도 같은 방법으로 정했다 - OTel Agent 전환을 검토하다 48시간 A/B 대신 E2E 트레이스 실측을 먼저 해 Kafka 경계를 넘어 traceId가 이어짐을 확인하고 Brave 유지 · A/B 미실행으로 단았다(ADR-001).

을 먼저 실행하지 않고 실제 요구사항 충족 여부를 측정한 뒤 결정했다는 점입니다.

2.2 검증 과정에서 확인한 계측 한계

다만 Trace 전체가 완벽하게 계측되는 것은 아니었습니다. 일부 외부 SDK 구간에 Span Gap이 남아 있고, 예를 들어 FCM 호출은 약 994ms가 걸리는데 해당 구간에 span이 없습니다. 그래서 이후로는 "Trace가 있다"와 "장애 원인을 판단하기에 충분한 Trace가 있다"를 다른 문제로 취급했고, 데이터의 존재가 아니라 Evidence로 활용 가능한 수준까지 전달됐는지를 검증 대상으로 삼았습니다.

3. 데이터는 있는데 Evidence가 사라지고 있었다

에이전트를 돌리자 조사 6회 연속으로 로그가 한 줄도 들어오지 않았습니다. 보통 이런 건 원인이 하나인데, 파 보니 독립된 결함이 들어있고 이후 같은 종류가 둘 더 나왔습니다.

결함	왜 근거가 사라졌나
{app=~"content auth chat"}	app 라벨은 Loki에 없고 값에도 -service 접미가 빠져 매칭 스트림 0개
logfmt level=~"ERROR WARN"	평문 Logback에서 level 필드가 만들어지지 않아, 선택터를 고쳐도 이 쿼리만 빈 결과
~ "ERROR WARN"	헤더 줄만 통과시키고 스택 프레임을 배제 - 정답이 스택에만 있는 문항의 근거가 전량 소실
parentSpanId 미파싱	span은 다 있는데 누가 누구를 불렀는지 모르는 상태 - 호출 구조가 데이터에 있어도 개별 span으로만 보임

특히 스택 트레이스 손실이 치명적이었습니다. 원본 Log에는 Root Cause가 있는데 에이전트가 받은 Evidence에는 없는 상태가 만들어졌기 때문입니다. parentSpanId는 새로 수집한 것이 아니라 이미 존재하는 관계 정보를 복원한 것입니다. 이후 감점이 발생하면 에이전트의 판단 문제로 단정하지 않고, "백엔드에 실제 데이터가 존재하는가 → 쿼리가 가져왔는가 → 필터가 제거하지 않았는가 → Collector가 변환에서 버리지 않았는가 → 최종 Evidence에 도달했는가"를 역순으로 확인하는 절차를 채점 규칙에 넣었습니다. Observability Backend에 데이터가 존재하는 것과 에이전트가 Evidence로 쓸 수 있는 것은 다릅니다.

4. 장애가 있었는데 못 찾았다 - 4에서 100까지

4.1 1초 차이

MongoDB를 23초 세우는 장애를 주입했습니다. 주입은 사양대로 났고 스위치 창도 주입을 포함했는데, 탐

색이 창을 좁히면서 장애 시작 1초 전에서 끊었습니다(장애 05:22:01~05:22:38Z ↔ 좁힌 창 끝 05:22:00Z). 결과는 4 / 100.² 처음에는 원인이 명확해 보였습니다 - "창이 좁아서 놓쳤다."

4.2 채널별 역추적 - 가설이 틀렸다

규칙대로 채널 원본을 직접 열어 보니 반대였습니다. Trace는 {status=error}에 05:22가 없고(에러가 아니라 23초 지연이었습니다), Log는 ERROR|WARN 기준으로 판별력이 없었으며, Metric은 step=5m이라 단기 장애를 포착하지 못했습니다. 즉 창을 다시 넓혀도 장애 후보를 만들 Signal 자체가 부족했습니다. 부족한 것은 범위가 아니라 채널 종류와 해상도였고, 커버리지는 둘의 곱이라는 것을 여기서 배웠습니다.

4.3 두 종류의 개선 - 4 → 85 → 100

- 지연 트레이스 채널(slow) 신설 - 후보 10 건이 전부 이 채널에서 나왔고 그중 2 건이 정답. 4 → 85.³
- 프롬프트 한 문장 추가 - "대기 시간을 타임아웃 상한과 대조하고 최종 상태를 명시하라." 코드 · 수집 · 창 무변경으로 85 → 100. 없던 절이 생겼습니다 - "상한 30,000ms ↔ 실측 22.5s → 만료되지 않음".

하나의 장애 안에 서로 다른 두 문제가 있었습니다. Signal이 부족한 문제는 Collector · Query · 탐색을 고쳐야 하고, Signal이 있는데 판단하지 못한 문제는 프롬프트를 고쳐야 합니다. 처방이 정반대이므로, 탐색과 판단을 분리한 구조가 실제로 개선 방향을 결정해 줬습니다.

5. Context 는 양이 아니라 밀도다

5.1 창을 2.26 배 늘렸더니

같은 문항을 두 번 돌렸는데 탐색이 고른 창 길이만 달라 자연 실험이 됐습니다(10분 43초 → 24분 16초, ×2.26). Context는 +72.2%, 비용은 +38.7%였지만 점수는 100 → 95로 오르지 않았습니다. "Context를 늘려도 정확도는 안 오른다"로 끝내고 싶지 않아 증가분을 채널별로 분해했습니다. 늘어난 것은 대부분 Metric이었습니다 - 8종 × 98포인트 × 다중 시리즈로 약 164,988B, 반면 Trace는 18,908B, Log는 16줄이었습니다. 정확한 결론은 "늘어난 Context 대부분이 정답 판단과 관계없는 채널이었다"입니다. 문제는 양이 아니라 Evidence 밀도였습니다.

5.2 줄이기 전에 실제 사용량부터 잴다

² 두 회차는 4분 간격 · 같은 주입 데이터라 완전히 독립적이지 않다. 재현된 것은 "같은 조건에서 결정적으로 4"이며 인용 시 함께 밝힌다.

³ 반증 조건도 함께 적어 줬다 - 필터를 뺐더니 무관 트레이스가 후보를 잠식하거나 정답이 수집 상한에 밀리면 원인은 필터가 아니라 상한이다.

비용을 분해하면 탐색 2% / 분석 98%였고, 분석 입력은 Trace 47% · Log 42%였습니다. 처음에는 Loki step을 5m → 1m으로 줄여 Log를 깎으려 했는데, 크기는 그대로였고 비용이 약 3% 늘었습니다. 원인은 limit=1000 - 창과 무관하게 항상 1,000줄이 들어오기 때문에 창 조절만으로는 Log Context가 줄지 않습니다. 그래서 버릴 것을 고르는 대신, 과거 RCA Report 44건을 역추적해 실제로 인용된 코드 위치를 확인했습니다.

로그 데이터	공급 비중	리포트가 실제 인용	조치 결과(저장 원본 전수)
라이브러리 프레임	71% (903줄)	0회	로그 3,237,421B → 1,390,968B (-57.0%)
애플리케이션 프레임	3.7% (50줄)	32회 / 12개 클래스	트레이스 1,863,242B → 1,305,152B (-30.0%)

가장 많은 자리를 차지하는 라이브러리 프레임은 한 번도 인용되지 않았고, 3.7%뿐인 애플리케이션 프레임이 반복적으로 근거였습니다. 그래도 지우지 않고 접었습니다(Fold) - 쓰이지 않았다는 것이 앞으로 필요 없다는 뜻은 아니기 때문입니다. 애플리케이션 프레임 · 예외 헤더 · 예외 메시지의 수치 값 · 경계 ±1 프레임 · 타임스탬프와 반복 패턴은 보존 규칙으로 명시하고 단위 테스트로 고정했습니다. Trace 도 삭제 대신 반복 attribute를 공통 영역으로 끌어올리고 표현을 평탄화해 압축했습니다.

단, 효과를 과장하지 않았습니다. 위 수치는 저장 원본 전수 기준이고, Fold를 적용한 실행에서 Context Token도 줄었지만 창과 장애 조건이 달랐기 때문에 Token 감소 전체를 Fold의 순수 효과로 귀속하지 않았습니다. 동일 창 · 동일 장애를 고정한 A/B는 아직 수행하지 않았습니다. 측정할 수 없는 효과는 개선 효과라고 주장하지 않는다 - 이 프로젝트의 검증 원칙입니다.

6. 성능을 어떻게 믿을 것인가 - 채점 체계와 그 체계의 결함

AI 시스템은 평가 방법이 조금만 달라져도 "좋아진 것 같다"가 쉽게 만들어집니다. 그래서 에이전트보다 채점 체계를 먼저 세웠습니다.

원칙	내용
앵커 박제	기준 답안을 장애 주입 전에 확정. 결과를 보고 고치면 그 회차는 무효, 개정은 다음 회차부터 적용
블라인드	리포트 생성 경로에 정답지를 두지 않고, 리포트 안에서 점수를 만들게 하지 않는다(자기 채점 무효)
$N \geq 2 \cdot \text{편차} \pm 10$	문항당 최소 2회. 평균 ± 최대편차로 기록하고 편차 10 초과면 인용 보

원칙	내용
	류
채점 불가	주입이 성립하지 않았거나 앵커 요건의 대상이 부재하면 점수를 만들지 않는다 - 0점과 구별한다

그리고 이 자(尺) 자체가 세 번 틀렸습니다.

6.1 앵커와 루브릭이 틀렸다

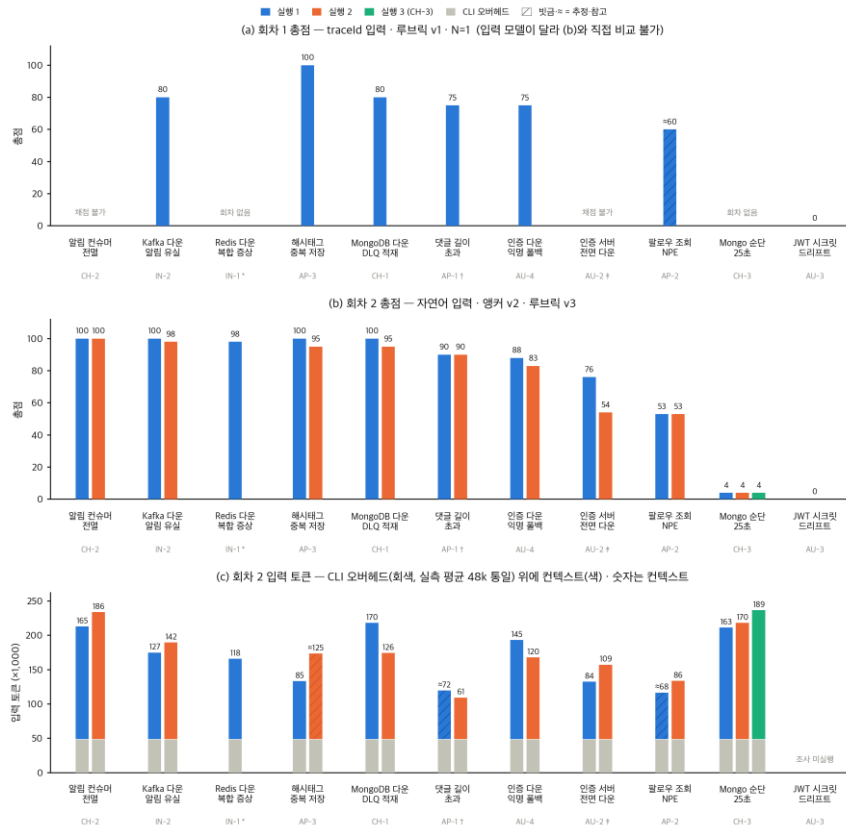
초기 채점 불가 4건이 전부 에이전트가 아니라 앵커의 결함이었습니다(부적합률 67% → 33%). 채점 항목도 v1(근본 40 / 근거 30 / 오귀인 20 / 조치 10)에서 v3(원인 40 / 근거 25 / 탐색 15 / 영향 10 / 오귀인 5 / 조치 5)로 바꿨습니다. 근본 40이 원인과 영향을 함께 담아 감점 전액이 영향 오판인 회차가 원인을 틀린 것처럼 읽혔고, 오귀인 20은 11회 전부 만점이라 척도가 아니었습니다. v3에서 중간 구간을 넣자 12회 만에 첫 비만점이 나왔습니다 - 문항이 아니라 자를 바꿔서 변별력이 생겼습니다. 그래서 v1과 v3 총점을 나란히 두고 개선이라 말하지 않습니다.

6.2 피험자가 시험지를 읽고 있었다

LLM CLI 서브프로세스가 JVM의 작업 디렉터리를 물려받는데, 그 CLI는 그 디렉터리의 프로젝트 문서를 자동으로 읽습니다. 평가 대상이 관측 라벨과 쿼리 결함이 적힌 문서를 읽은 상태로 시험을 보고 있었던 것입니다. 격리 디렉터리로 분리하고 오염 크기를 확인한 뒤(격리 26,603 ↔ 레포 루트 32,747 · +23%), 격리 전 회차는 전부 폐기하고 재실험했습니다.

6.3 토큰 지표 자체에 버그가 있었다

하네스가 실제 42,774 토큰을 2로 기록하고 있었고, 상수로 가정했던 CLI 오버헤드는 하루에 약 20% 움직였으며(22,000 → 26,626 → 21,247), 흔히 쓰는 chars/4 추정은 실측 대비 최대 2.1배 과소였습니다. 소급 추정이 불가능해 조사마다 직접 재는 프로브를 넣었습니다. 지표를 만들었다고 그 지표를 믿은 것이 아니라, 지표의 타당성을 따로 검증했습니다.



[그림 2] (a) traceld 입력 · 루브릭 v1 · N=1 / (b) 자연어 입력 · 앵커 v2 · 루브릭 v3 / (c) 입력 토큰 - 회색이 CLI 고정 오버헤드, 색이 실제 컨텍스트. (a)와 (b)는 자가 달라 직접 비교하지 않는다.

6.4 같은 자로 두 번 잰 값 - N=2 결과

루브릭이 회차별로 바뀌었으므로 서로 다른 루브릭의 총점을 비교하지 않고, 같은 자로 반복 측정한 동일 문항의 N=2 결과만 기록했습니다.

문항	N=2	무엇이 재현됐나
CH-2	100 ± 0	여섯 항목 전부 앵커 문언만으로 결정 · 판정 0건
IN-2	99 ± 1	갈린 하나가 후보 배치 형식 차이 · 판단은 동일
AP-3	97.5 ± 2.5	다섯 항목 편차 0, 갈린 하나가 영향 항목
CH-1	93 ± 7	편차의 원인이 능력이 아니라 채널 운(\$7)
AU-4	85.5 ± 2.5	원인 30이 다섯 회차 연속 같은 사유로 재현
AU-2	85 ± 0	이전 구성에서는 65 ± 11로 인용 보류였다
CH-3	4 ± 0	탐색이 주입을 1초 차이로 잘라 산출물이 없다(\$4)

편차 0이 곧 안정성은 아닙니다. CH-3의 0은 막힘의 재현, CH-2의 0은 신호가 전부 도달한 상태의 재현으로 뜻이 정반대인데 숫자는 같습니다. 그래서 채점표에 "정답 신호가 컨텍스트에 도달했는가"를 편차

와 함께 남기도록 했습니다. Top-1 적중률 · Evidence Recall · 탐색 정밀도는 값을 비워 뒀습니다 - 정상 구간 문항을 아직 0회 돌려 "없는 원인을 지어내지 않는가"를 켜 적어 없고, 정밀도 없는 재현율은 지표가 아니기 때문입니다.

7. 같은 조건인데 86 과 100 이 나왔다

단일 실행 점수를 성능으로 보면 안 되는 이유를 가장 명확하게 보여준 사례입니다. 같은 주입 데이터를 18분 간격으로 두 번 조사했습니다. 문안 · 도구 · 모델 전부 동일한데 86 ↔ 100이었습니다. LLM 편차로 결론 내리지 않고, 점수가 같린 Evidence를 역추적했습니다. 모델 편차로 접하려던 다음 행동이 안 나오므로 같린 항목을 따라가니, 12점이 트레이스 한 건에 걸려 있었습니다 - 발행 → 실패 4회 → DLQ 발행 → 재처리 성공이 한 traceId 안에 전부 들어 있는 그 트레이스입니다.

항목	실행 1 (86점)	실행 2 (100점)
durationMs	3,505,144,756 (40.5일 · 깨짐)	31,370 (31.4초 · 정상)
startTimeUnixNano · durationNanos	둘 다 정상	동일
후보 선정 / 그 traceId의 로 그	탈락 / 37줄 · 전부 헬스체크	선택 / 132줄 · DLQ 발행 1 · 재처리 성공 1 · 실패 4

적재 중 일시적으로 깨진 값이었고 18분 뒤 정상으로 돌아왔습니다. 문제는 코드가 주석과 반대로 동작한 것이었습니다 - 주석에는 "버리면 도달할 길이 없으므로 trusted=false로 남기고 정렬과 창 계산에 서만 뺀다"고 적혀 있는데, 실제로는 수집 후보 목록에서도 빼고 있었습니다. 깨진 필드 하나가 정답 트레이스를 통째로 컨텍스트에서 지운 것이고, 검증은 장애 주입 없이 됩니다 - 깨진 값이 든 원본 응답이 그대로 저장돼 있어 단위 테스트 입력으로 쓰고, 성공 기준은 "후보 목록에 그 traceId가 들어오는가" 하 나입니다.

같은 회차에서 로그 조회가 limit=1000 + direction=forward라 창 44.4분 중 앞 22.9분만 들어오고 뒤 34%가 잘린 것도 잡혔습니다 - 장애 조사에서 잘리는 뒤쪽은 언제나 복구 · 도착 구간입니다. 단일 실행 점수는 성능이 아닙니다. 같은 조건에서 한 필드의 일시적 파손이 14점을 만들었습니다. $N \geq 2$ 와 편차 기준, "감점마다 신호 존재를 직접 확인한다"는 규칙이 왜 필요한지를 이 사례가 증명합니다.

8. Engineering Findings 와 남은 것

8.1 확인한 것

1	RCA 성능의 상한은 LLM이 아니라 Signal Quality가 정할 수 있다 - 막힌 채널 수와 점수가 정렬됐다
---	--

	(0개 100 · 90 → 1개 88 → 2개 이상 76 · 53). 표본 5개라 상관은 주장하지 않고 다음 회차의 반증 대상으로 등록했다
2	Observability Data와 Evidence는 같지 않다 - 백엔드에 있어도 쿼리 · 필터 · 상한 · 해상도 · Collector 어디서든 사라진다
3	Context는 양이 아니라 정답과 관계있는 신호의 비중 - 창 ×2.26에 컨텍스트 +72.2%, 점수는 그대로 였고 늘어난 것은 메트릭이었다
4	탐색과 판단은 분리하는 편이 낫다 - 못 찾은 것과 찾기도 못 푼 것은 처방이 정반대라, 한 점수로 합치면 무엇을 고칠지 정할 수 없다
5	측정 지표와 단일 실행 점수는 그 자체로 검증 대상이다 - 상수인 줄 알았던 값이 하루에 20% 움직였고, 동일 조건에서 한 필드 파손이 86 ↔ 100을 만들었다

실측으로 기각한 가설들도 그대로 남깁니다. "창이 좁아서 못 찾았다" → 신호 자체가 없었다. "도구를 고치면 +25점" → -10점, 결함이 둘 더 있었다. "절감이 안 나온 건 창 탓" → 트레이스 채널만 접기가 안 걸려 있었다. "메트릭 처방이 틀렸다" → 처방은 맞고 하한 숫자가 틀렸다(15 → 30초).⁴ "오버헤드는 상수다" → 하루에 20% 움직였다.

8.2 현재 한계

- 미계측 SDK 구간 - FCM 994ms 가 span 없이 빈다.
- 정상 문항 0 회 - Precision · False Positive · Top-1 · Evidence Recall 을 아직 완성된 지표로 제시하지 않는다.
- Context 최적화 A/B 미완료 - 저장 원본 감소는 측정했지만 동일 장애 · 동일 창에서의 순수 Token 절감 분리는 미수행.
- 거짓 신호 - Redis 가 죽었는데 `up{job="redis"}`는 1 이었다(익스포터 생사이지 Redis 생사가 아니다). 에이전트가 이 모순을 자력으로 잡아 확신도를 낮쳤고, 이는 알람 설계의 함정이기도 하다.

8.3 Next

우선순위는 깨진 행 필터 제거 · 로그 절삭 교정 · 반복 span 접기(셋 다 새 주입 없이 저장된 원본으로 검증된다) → 신호 공급 보강(redis_up · kube-state-metrics) → 정상 문항 투입으로 정밀도 · Top-1 개통 → Context 최적화 A/B → Code-aware RCA(Git diff · 배포 정보 연동 - 근거는 이미 확보된 (c)유형 실패) 순입니다.

⁴ 채점 기록에 "처방이 틀렸다고 적었다가 철회함"으로 남아 있다 - 판단이 바뀐 과정을 지우지 않는 것이 이 프로젝트의 문서 규칙이다.

8.4 마무리

점수를 높게 보이게 만드는 것보다, 측정하지 않은 것을 측정하지 않았다고 쓰는 것을 우선했습니다. 그래서 이 문서에는 Top-1 적중률도 Before/After 총점 비교도 없습니다. 있는 것은 같은 자료 두 번 이상 켜 값과 그 값의 한계뿐입니다. 이 프로젝트에서 얻은 가장 큰 것은, RCA 모델을 잘 만드는 것만큼 그 결과를 믿을 수 있도록 관측 · 평가 · 측정 시스템을 함께 설계하는 일이 중요하다는 사실이었습니다.

근거 docs/scoring/(채점 대장) · docs/changes.md(변경 ID 색인) · docs/round-5-6/. 모든 수치는 이 기록과 대조했다.